ScoreControl Backend: Status and Next Steps

Prepared by Guy Nutman (backend lead) for the team meeting · October 22, 2026

Login is finished on the backend and merged into **main**; nobody needs to reimplement it. The database schema (PR #2) is ready to merge. What's left: five technical decisions, connecting the frontend pages, moving development onto local Supabase, and setting up the hosted project for real users.

1. Where we are

Area	Status	Owner
Login backend (PR #1)	Merged into main Oct 21. Signup, login, logout, password reset, email confirmation, usernames, rate limiting. 197 of 198 tests pass; the one failure is a known flaky test.	Oliver
Database schema (PR #2)	Ready to merge into main. Defines users, projects, branches, commits, files, and pull requests. No migration yet, so the database itself is unchanged.	Guy
Login and signup pages	UI built on the ui/ux-designs branch. Not connected to the backend yet.	Ryan

2. Technical decisions needed

Details for each are in docs/DATABASE_SCHEMA.md. Write the answers in the last column.

#	Question	Context	Decision
1	Main file format: MusicXML or MuseScore?	The schema supports both. The choice drives the parser and diff engine.	
2	Which LLM outputs do we store?	E.g. commit message suggestions, "what changed" summaries. Stored output needs a column or small table.	
3	Profile creation trigger	Proposal: a Supabase trigger creates each user's public profile and copies their username at signup. Does it fit the auth design, and who writes it?	
4	Size limits	Max size per uploaded file, and max files per project.	
5	Archived branches	Add an archived_at column so a branch can be hidden without being deleted?	

3. Keys and secrets

Supabase gives us two kinds of keys, and they need opposite handling:

Value	What it can do	How to share
Project URL, publishable key (sb_publishable_...)	Only what Row Level Security allows (section 4). Designed to be public: it ends up in the browser.	Anywhere, including Slack
Secret key (sb_secret_...)	Full access to all data, ignoring Row Level Security. Server only.	Password manager only
DATABASE_URL, DIRECT_URL	Full database access: they contain the database password.	Password manager only

Why not Slack for secrets

- Messages are kept indefinitely, searchable by anyone in the workspace, and synced to every phone and laptop.
- Everyone who joins later can read old messages, and so can installed apps, bots, and workspace exports.

- Anyone who ends up with the secret key can read, change, or delete all of our data. Unlike a password, we might never notice it leaked.

Use a shared password manager vault (1Password, Bitwarden) or a one-time link that expires after it's opened. Better still, use local Supabase for development (section 5): then nobody needs the hosted project's secrets at all.

Where the secret key is

Supabase dashboard > **Project Settings > API Keys > Secret keys**. Create one key per person or environment (e.g. "ryan-dev", "production") with **New secret key**. If a laptop is lost, revoke just that key on the same page.

4. Row Level Security (RLS)

RLS is Postgres's per-row access control. With RLS on, anyone using the publishable key sees only the rows a policy allows; with RLS on and no policies, they see nothing. Supabase's warning means exactly this: the publishable key is only safe if every table it can reach has RLS on.

Status today: safe

- The only table we've created is the private username registry (scorecontrol_auth.usernames). It has RLS enabled with no policies, and its schema isn't exposed through Supabase's API.
- The three username lookup functions can only be called with the secret key; the publishable key can't call them.
- The schema in PR #2 hasn't created any tables yet (no migration), so there's nothing else to protect.

Rule from now on

Every migration that creates a public table must also enable RLS on it. Our app reaches the database through Prisma as the database owner, which RLS doesn't restrict, so turning RLS on with no policies blocks outside access without breaking the app. To check the hosted project, run this in the Supabase SQL Editor; every row should say true:

```
select schemaname, tablename, rowsecurity
from pg_tables
where schemaname in ('public', 'scorecontrol_auth');
```

5. Onboarding: local Supabase

Everyone runs their own private copy of Supabase in Docker. Nobody needs hosted secrets, experiments can't break shared data, and the keys it prints only work on your own machine. Keep the hosted project for the shared demo and testing.

1. **Install Docker Desktop** and keep it running. On Windows, enable WSL 2 integration in its settings.
2. **Get the code:** `git pull` on main, then `npm install`.
3. **Start Supabase:** `npx supabase start`. The first run downloads several GB of images. It applies everything in supabase/migrations, including the username migration.
4. **Get your keys:** `npx supabase status` prints the URLs, a publishable key (sb_publishable_...), and a secret key (sb_secret_...).
5. **Create .env.local** (`cp .env.example .env.local`) with the values below.
6. **Run the app:** `npm run dev`, then open http://localhost:3000 (not 127.0.0.1).

```
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<sb_publishable_... from supabase status>
SUPABASE_SECRET_KEY=<sb_secret_... from supabase status>
DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:54322/postgres
DIRECT_URL=postgresql://postgres:postgres@127.0.0.1:54322/postgres
APP_ORIGIN=http://localhost:3000
```

Command or URL	What it does
`npx supabase start` / `stop`	Start or stop your local Supabase (data is kept between runs)
`npx supabase db reset`	Wipe local data and re-apply all migrations

Command or URL	What it does
`npx prisma migrate dev`	Apply app table migrations to the database in DIRECT_URL (once we have them)
http://127.0.0.1:54323	Supabase Studio: browse tables and run SQL
http://127.0.0.1:54324	Mailpit: every email local Supabase sends (password resets, confirmations)

- Email confirmation is off locally, so new accounts can sign in right away.
- Never run `npx supabase db push` or `supabase config push` unless you mean to change the hosted project.
- Prisma commands use whatever DIRECT_URL is in .env.local, so check it points at 127.0.0.1 before running migrations.

6. Frontend: connecting the login pages (Ryan)

- Merge main into ui/ux-designs (`git merge origin/main`, then `npm install`). No conflicts.
- Connect the forms with fetch: `POST /api/auth/login` with `{ identifier, password, rememberMe }`, and `POST /api/auth/signup` with `{ email, password, username }`. Don't call Supabase from browser code.
- Change the login field to type="text" so usernames are accepted (type="email" rejects them).
- Restore the exact pin "next": "16.4.2" (no ^). Delete package-lock 2.json and skills-lock.json.
- Build /reset-password, the forgot-password form, and the username form. Spec: "Frontend integration" in docs/AUTH_API.md.

7. Before login works for real users

- Confirm Oliver's username migration is applied to the hosted project (dashboard > Database > Migrations).
- Fix the flaky test at tests/auth/supabase-sdk.test.ts line 196.
- Supabase dashboard: Site URL, redirect allowlist, and email templates, as listed in docs/ENVIRONMENT.md.
- Set up a custom email provider (e.g. Resend or Postmark). Supabase's built-in email sends only a few emails an hour, and only to team members.
- Turn on email confirmation for production.
- Pick hosting (e.g. Vercel) and set APP_ORIGIN, the Upstash Redis keys, and AUTH_TRUSTED_IP_HEADER there.

8. Database schema: next steps

- Apply the decisions from section 2 to the schema.
- Create the first migration (`npx prisma migrate dev --name init_schema`), with RLS enabled on every new table.
- Add the profile creation trigger (decision 3).
- Create a private Supabase Storage bucket for score files.
- Add the shared database client (src/lib/db.ts) and turn the schema checks into an automated test suite.